

LUỒNG LÀM VIỆC MẪU (Workflow Examples)

Tài liệu minh họa cách các AGENT phối hợp giải quyết các tình huống thực tế. Đọc cùng `RULES.md` để hiểu đầy đủ.

Ví dụ 1: Yêu cầu tính năng mới "Đăng nhập bằng Google"

```
sequenceDiagram
    participant U as User/Stakeholder
    participant PM as Product Manager
    participant BA as Business Analyst
    participant UX as UI/UX Designer
    participant EM as Engineering Manager
    participant PJM as Project Manager
    participant TL as Tech Lead
    participant SD as Senior Developer
    participant JD as Junior Developer
    participant QAL as QA Lead
    participant QA as QA Engineer
    participant DOL as DevOps Lead
    participant DO as DevOps Engineer
    participant CTO as CTO
    participant UXR as UX/UI Reviewer

    U->>PM: "Cần đăng nhập bằng Google"
    PM->>PM: Viết PRD (vì sao, mục tiêu, metric)
    PM->>BA: Bóc tách user story
    BA->>PM: User story + AC + câu hỏi mở
    PM->>UX: Yêu cầu mockup
    UX->>PM: Mockup + spec
    PM->>EM: Trình bộ tài liệu để estimate
    EM->>CTO: Thông báo (vì liên quan auth - bảo mật)
    CTO->>EM: Approve + lưu ý OAuth flow
    EM->>TL: Giao thiết kế kỹ thuật
    TL->>TL: Viết technical design doc
    TL->>CTO: Review kiến trúc OAuth
    CTO->>TL: Approve
    TL->>PJM: Cung cấp task breakdown
    PJM->>PJM: Lên sprint
    TL->>SD: Giao task "OAuth backend integration"
    TL->>JD: Giao task "UI login button + redirect handler"
    SD->>SD: Code OAuth backend
    JD->>SD: Hỏi cách handle callback
    SD->>JD: Mentor, pair programming
    JD->>SD: PR review
    SD->>TL: Approve, escalate review
    TL->>TL: Review cuối, merge
    TL->>QAL: Báo feature ready cho staging
    QAL->>QA: Giao test plan
    QA->>QA: Manual + automation test
    QA->>SD: Log bug "không handle khi user reject scope"
    SD->>QA: Fix + push
    QA->>QA: Verify, regression pass
```

Note over TL,UXR: Feature có UI login button mới → chèn UX/UI Reviewer TRƯỚC sign-off

TL->>UXR: Yêu cầu kiểm tra trực quan (đã đổi/thêm giao diện)
UXR->>UXR: Chạy app thật, chụp screenshot, đánh giá C1-C7
UXR->>QAL: Report – Pass / issue cần fix
QAL->>EM: Sign-off chất lượng
EM->>DOL: Yêu cầu deploy production
DOL->>DO: Deploy production
DO->>DOL: Deploy success, monitor OK
DOL->>EM: Confirm released
EM->>CTO: Report release done

Bài học từ ví dụ này:

- KHÔNG ai nhảy cấp. PM không nói thẳng với Junior Developer.
- CTO can thiệp vì liên quan đến bảo mật (OAuth), nhưng chỉ approve ở mức kiến trúc.
- Junior hỏi Senior (mentor), Senior báo lên Tech Lead.
- QA có quyền VETO. Không sign-off thì không deploy.
- UX/UI Reviewer chèn vào TRƯỚC QA Lead sign-off vì feature có UI login button mới — bỏ qua bước này nếu feature chỉ đổi backend/logic.

Ví dụ 2: Bug Production khẩn (SEV1)

sequenceDiagram

participant Alert as Monitoring Alert
participant DO as DevOps Engineer
participant DOL as DevOps Lead
participant TL as Tech Lead
participant SD as Senior Developer
participant EM as Engineering Manager
participant CTO as CTO
participant PM as Product Manager
participant QA as QA Engineer

Alert->>DO: SEV1 alert: API 500 rate 80%
DO->>DO: Ack trong 5 phút
DO->>DOL: Page DevOps Lead
DOL->>EM: Page Engineering Manager (SEV1)
DOL->>CTO: Page CTO (SEV1)
DOL->>DO: Mở incident channel #inc-001
DOL->>TL: Cần Tech Lead support
TL->>SD: Kéo Senior Dev vào incident
SD->>SD: Investigate - tìm root cause
DOL->>DOL: Decide rollback hay hot-fix?
DOL->>DO: Rollback về version trước
DO->>DOL: Rollback OK, alert dừng
DOL->>EM: Mitigate xong, đang find root cause
SD->>TL: Tìm ra: race condition trong cache layer
TL->>SD: Viết fix + test
SD->>TL: PR
TL->>TL: Review nhanh, urgent
TL->>QA: Cần test gấp trên staging
QA->>QA: Test fix, OK
QA->>QAL: Sign-off hot-fix

```
DOL->>DO: Deploy hot-fix production
DO->>DOL: Deploy OK
DOL->>EM: Resolved
DOL->>DOL: Viết post-mortem (48h)
PM->>PM: Update user về incident (nếu cần)
CTO->>EM: Yêu cầu review process để tránh lặp lại
```

Bài học:

- Trong incident, **bỏ qua chain of command** (escalate thẳng lên CTO theo Quy tắc 6 của **RULES.md**).
- **Mitigate trước, fix root cause sau**. Rollback là an toàn nhất.
- QA vẫn phải test fix dù gấp - không bỏ bước.
- Sau incident, **BẮT BUỘC** có post-mortem.

Ví dụ 3: Junior Developer bị block

```
sequenceDiagram
    participant JD as Junior Developer
    participant SD as Senior Developer
    participant TL as Tech Lead
    participant EM as Engineering Manager

    JD->>JD: Code task, gặp lỗi không hiểu
    Note over JD: Tự thử trong 30 phút<br/>(đọc doc, search, đọc code)
    JD->>SD: Hỏi Senior (mentor)
    Note over JD: Format câu hỏi tốt:<br/>bối cảnh, đã thử gì, câu hỏi cụ thể
    SD->>JD: Giải thích pattern, pair programming
    JD->>JD: Tiếp tục code
    JD->>JD: Vẫn không xong sau 1 ngày
    JD->>TL: Escalate (qua daily report hoặc trực tiếp)
    TL->>TL: Đánh giá: task có nằm trong khả năng Junior?
    TL->>JD: Quyết định 1: re-assign sang Senior + Junior pair
    TL->>SD: Pair với Junior để cùng làm
    Note over TL: Đây không phải failure<br/>của Junior - đây là<br/>điều chỉnh resource
```

Bài học:

- Junior **PHẢI** tự thử 30 phút trước khi hỏi (không phải lười).
- Hỏi đúng format (bối cảnh + đã thử + câu hỏi cụ thể).
- Tech Lead có thể re-assign mà không phải đổ lỗi Junior.

Ví dụ 4: Conflict giữa PM và Tech Lead

```
sequenceDiagram
    participant PM as Product Manager
    participant TL as Tech Lead
    participant EM as Engineering Manager
    participant CTO as CTO

    PM->>TL: "Cần feature X xong trong 1 tuần"
    TL->>PM: "Không khả thi - cần 3 tuần do phải refactor module Y"
    PM->>TL: "Cắt refactor, làm hack nhanh thôi"
    TL->>PM: "Hack sẽ tạo tech debt, ảnh hưởng feature sau"
```

```
Note over PM,TL: Conflict không giải quyết được trong 1 ngày
TL->>EM: Escalate
PM->>EM: Đồng thời escalate
EM->>EM: Nghe cả 2 phía
EM->>EM: Quyết định: cắt scope feature X<br/>(giữ refactor, làm phần lỗi)
EM->>PM: Thông báo quyết định + lý do
EM->>TL: Thông báo quyết định + lý do
Note over PM,TL: Nếu PM vẫn không đồng ý:<br/>escalate lên CTO
PM->>CTO: Escalate (qua EM, kèm CC)
CTO->>CTO: Quyết định cuối cùng
```

Bài học:

- Conflict không giải quyết được trong 1 ngày → escalate (Quy tắc 6).
- Escalate phải đi qua cấp trên trực tiếp, có CC.
- Quyết định cuối thuộc về cấp cao hơn, các bên phải tuân thủ.

Ví dụ 5: Code Review từ Junior → Senior → Tech Lead

```
flowchart TD
    JD[Junior Dev tạo PR] --> A{Self-check}
    A -->|Có test, có doc| B[Tag Senior Dev review]
    A -->|Thiếu| JD
    B --> SD[Senior Dev review]
    SD -->|Có vấn đề lớn| C[Comment, request changes]
    C --> JD
    SD -->|OK| D[Senior approve]
    D --> E[Tag Tech Lead review cuối]
    E --> TL{Tech Lead review}
    TL -->|Có vấn đề| C
    TL -->|OK & PR thường| F[Merge]
    TL -->|OK & PR critical| G[Cần EM approve]
    G --> EM[Engineering Manager review]
    EM -->|OK| F
    EM -->|Cần CTO| CTO[CTO approve]
    CTO --> F
```

Bài học:

- 2-eyes principle: KHÔNG ai self-merge.
- PR thường: Senior → Tech Lead là đủ.
- PR critical (auth, payment, DB schema): cần thêm EM, có khi CTO.

Ví dụ 6: Sprint Planning chuẩn

```
sequenceDiagram
    participant PM as Product Manager
    participant BA as Business Analyst
    participant TL as Tech Lead
    participant PJM as Project Manager
    participant QAL as QA Lead

    PJM->>PM: Mời PM chuẩn bị backlog ưu tiên
    PM->>BA: Đảm bảo top story có AC rõ
```

```
BA->>PM: Confirm ready
PJM->>TL: Mời TL chuẩn bị technical estimate
TL->>TL: Pre-estimate trước họp

Note over PM,QAL: Họp Sprint Planning (2-4h)
PM->>PJM: Trình bày top 10 story ưu tiên
TL->>PM: Hỏi làm rõ scope
BA->>PM: Làm rõ AC
TL->>PJM: Estimate từng story
QAL->>TL: Yêu cầu testability cho mỗi story
PJM->>PJM: Chốt sprint backlog (theo velocity team)
PM->>PM: Đồng ý cut-off line

PJM->>PJM: Tạo task trong board
TL->>PJM: Assign task cụ thể cho Senior/Junior
QAL->>PJM: Assign test task cho QA Engineer
```

Bài học:

- Sprint planning phải có ĐÚ: PM, BA, TL, PJM, QA Lead.
- Story phải READY trước họp (có AC rõ).
- Estimate là việc của Tech Lead, không phải PM.
- Cut-off scope dựa trên velocity, không phải mong muốn.

Ví dụ 7: Luồng Fast-Track sửa lỗi UI nhỏ (WF-FASTTRACK)

Tình huống: Người dùng yêu cầu "Sửa lại màu nút Đăng nhập bị sai mã màu và sai chính tả thành 'Đăng Nhập'".

```
sequenceDiagram
    participant U as User
    participant Disp as Dispatcher
    participant JD as Junior Developer
    participant TL as Tech Lead
    participant QA as QA Engineer
    participant DO as DevOps Engineer
    participant UXR as UX/UI Reviewer

    U->>Disp: "Sửa màu và typo nút Đăng nhập"
    Disp->>Disp: Đánh giá: P3, UI thay đổi nhỏ, không đổi Logic
    Disp->>Disp: Kích hoạt WF-FASTTRACK
    Disp->>JD: Giao thẳng task

    JD->>JD: Khởi tạo PROGRESS.md (Trạng thái: TỰ PHÊ DUYỆT)
    Note over JD: KHÔNG chờ User gõ APPROVE PLAN
    JD->>JD: Đọc CODE-GRAPH.md tìm Component Login
    JD->>JD: Sửa code CSS và Typo
    JD->>JD: Đánh dấu DONE trong PROGRESS.md
    JD->>TL: Tạo PR khẩn cấp

    TL->>TL: Review nhanh (Chỉ check giao diện/Typo)
    TL->>TL: Merge PR

    Note over TL,UXR: Fast-Track đổi màu + typo UI → BẮT BUỘC UX/UI Reviewer trước khi QA
    TL->>UXR: Kiểm tra trực quan nhanh (đã đổi giao diện)
    UXR->>UXR: Chụp screenshot nút Đăng nhập, so màu (#F05922) + đúng chữ
```

UXR->>TL: Pass – đúng màu, đúng chính tả

TL->>QA: Báo cáo Smoke Test

QA->>QA: Mở app, check đúng nút Đăng nhập

QA->>DO: Pass Smoke Test, cho phép Deploy

DO->>DO: Deploy Staging/Production

DO->>U: Báo cáo hoàn thành Fast-Track

Ví dụ 8: WF-MIGRATE — Chuyển đổi Framework (Code Migrator)

Tình huống: Người dùng yêu cầu "Chuyển project IPGSUseCam từ WinForms sang Avalonia, chạy được cả Windows và Linux".

sequenceDiagram

participant U as User
participant Disp as Dispatcher
participant CM as Code Migrator
participant SD as Senior Developer
participant JD as Junior Developer
participant QA as QA Engineer
participant QAL as QA Lead

U->>Disp: "Chuyển WinForms → Avalonia, chạy được Windows + Linux"

Disp->>Disp: Yêu cầu rõ ràng migrate framework → kích hoạt WF-MIGRATE

Disp->>CM: Giao task (Opus)

CM->>CM: Lessons Check (avalonia/ + csharp-winforms/ + dotnet-general/)

CM->>CM: Cấp 0 – Glob/Grep đếm N file nguồn, M control/timer/event

CM->>CM: Cấp 1-2 – Lập bảng inventory chi tiết (khớp N/M)

CM->>CM: G2 – Lập 3 bảng mapping (component, pattern, kiểu dữ liệu)

CM->>CM: S2A(c) – Rà toàn bộ dependency, đánh giá Linux-OK

CM->>CM: Lập plan có nhóm song song

CM->>U: Trình plan + inventory + mapping – xin duyệt

U->>CM: Duyệt plan

CM->>SD: Giao task UI/logic phức tạp (Sonnet)

CM->>JD: Giao task CRUD/UI đơn giản (Sonnet)

SD->>CM: Nộp artifact + build sạch (win-x64 + linux-x64)

JD->>CM: Nộp artifact + build sạch

CM->>CM: Review (correctness > behavior parity > security > style)

CM->>CM: G6 – Đối chiếu 100% inventory + re-check dependency lần cuối

CM->>CM: Publish thử linux-x64 – BẮT BUỘC pass

CM->>QA: Smoke test path chính, đối chiếu behavior parity

QA->>QAL: Sign-off (P0/P1 phải sạch)

QAL->>U: Migration hoàn thành – báo cáo inventory/dependency đầy đủ

Bài học từ ví dụ này:

- Code Migrator KHÔNG tự code hàng loạt — chỉ khảo sát/lập plan/review (Opus), giao việc code thực tế cho Senior/Junior Dev (Sonnet).
- Inventory PHẢI khớp số đếm thực tế (Cấp 0) — không liệt kê mẫu, tránh bỏ sót tính năng.
- Dependency PHẢI được rà lại lần cuối ở G6 (không chỉ 1 lần ở đầu) — bắt các package Senior/Junior Dev thêm giữa đường.

- Workflow này KHÔNG tự động kích hoạt — chỉ khi user yêu cầu rõ ràng chuyển đổi framework/ngôn ngữ.

Ví dụ 9: Song song hoá — QA Engineer // UX/UI Reviewer (lấy cảm hứng từ Ruflo/Claude Flow)

Tình huống: Feature "Đăng nhập bằng Google" (tiếp theo Ví dụ 1) vừa được Tech Lead merge, có cả logic OAuth và UI login button mới. Thay vì chạy UX/UI Reviewer RỒI MỚI đến QA Engineer (tuần tự), Dispatcher gọi cả hai chạy đồng thời vì cả hai cùng nhận code đã merge và không phụ thuộc lẫn nhau (đủ điều kiện theo `RULES.md §3.4`).

```
sequenceDiagram
    participant TL as Tech Lead
    participant Disp as Dispatcher
    participant QA as QA Engineer
    participant UXR as UX/UI Reviewer
    participant QAL as QA Lead

    TL->>Disp: Code đã merge — có đổi UI + logic
    Disp->>Disp: Kiểm tra điều kiện RULES.md §3.4 — QA & UXR độc lập, cùng nhận input từ TL
    par Chạy song song (1 lời gọi Agent tool)
        Disp->>QA: Test chức năng (functional)
        QA->>QA: Manual + automation test
    and
        Disp->>UXR: Đánh giá trực quan (C1-C7)
        UXR->>UXR: Chạy app thật, chụp screenshot
    end
    QA->>QAL: Report functional test
    UXR->>QAL: Report UX review
    QAL->>QAL: Tổng hợp cả 2 báo cáo — Sign-off
```

Bài học từ ví dụ này:

- Song song hoá CHỈ áp dụng khi 2 bước thật sự độc lập (`RULES.md §3.4`) — QA và UXR cùng nhận 1 input (code merge), không bên nào chờ kết quả bên kia.
- Rút ngắn thời gian workflow đáng kể cho cặp bước này so với chạy tuần tự, mà KHÔNG bỏ bớt bất kỳ bước kiểm tra nào.
- QA Lead vẫn phải nhận ĐỦ cả 2 báo cáo trước khi sign-off — không sign-off khi thiếu 1 trong 2.
- KHÔNG áp dụng song song cho cặp có quan hệ review (VD: Senior Dev code → Tech Lead review vẫn PHẢI tuần tự).

Ví dụ 10: WF-GITHUB-RESEARCH (Mode A) — Nghiên cứu repo GitHub để đề xuất cải tiến KZTEK

Tình huống: Người dùng gửi "Nguồn: <https://github.com/addyosmani/agent-skills> — nghiên cứu github này giúp tôi, sau đó đề xuất cải tiến".

```
sequenceDiagram
    participant U as User
    participant Disp as Dispatcher
```

```
participant GRR as GitHub Repo Researcher
participant TL as Tech Lead

U->>Disp: Gửi link GitHub + yêu cầu nghiên cứu
Disp->>Disp: Có link GitHub → kích hoạt WF-GITHUB-RESEARCH
Disp->>GRR: Giao task (Sonnet)

GRR->>GRR: Bước 0 – Phase 0 Audit (kiểm tra nhánh/plan/artifact đã có chưa)
GRR->>GRR: Bước 1 – Tạo nhánh research/<repo-slug>-<date>
GRR->>GRR: Bước 2 – Clone repo vào scratchpad (ngoài working tree), đọc & phân tích
GRR->>U: Bước 3 – Viết phân tích repo (mục đích, cấu trúc, điểm nổi bật) – KHÔNG
kèm đề xuất
Note over GRR,U: User đã nói rõ mục đích "đề xuất cải tiến" → Mode A
GRR->>U: Bước 3b – Viết bảng đề xuất cải tiến riêng biệt (dựa trên Bước 3), trình
user
U->>GRR: Bước 4 – Chọn đề xuất được áp dụng
GRR->>GRR: Bước 4b – Áp dụng đề xuất đã chọn, commit lên nhánh nghiên cứu
opt Đề xuất đúng kiến trúc/logic nghiệp vụ đáng kể
    GRR->>TL: Khuyến nghị review nhanh trước khi merge
    TL->>GRR: Review xong
end
GRR->>U: Bước 5 – Xin xác nhận merge về main
U->>GRR: Xác nhận merge
GRR->>GRR: Bước 5b – Merge về main, báo cáo kết quả
```

Bài học từ ví dụ này:

- GitHub Repo Researcher CHỈ kích hoạt khi user gửi link GitHub — không tự động chạy trong workflow khác.
- Repo ngoài clone về thư mục scratchpad để đọc, không đưa `.git` của repo ngoài vào commit KZTEK.
- **Bước 3 và 3b tách bạch hoàn toàn:** Bước 3 chỉ là phân tích trung lập (mô tả repo, không recommend); Bước 3b mới là đề xuất cải tiến dựa trên phân tích đó. Tách bạch giúp user đánh giá khách quan trước khi bị "push" vào đề xuất.
- Không tự áp dụng đề xuất (Bước 4b) khi user chưa chọn ở Bước 4; không tự merge (Bước 5b) khi user chưa xác nhận rõ ràng tại đúng thời điểm merge — tuân thủ Git Safety Protocol chung của hệ thống.
- Nếu thay đổi đúng kiến trúc/logic nghiệp vụ đáng kể → khuyến nghị Tech Lead review trước khi merge (Two-Eyes Principle).

Ví dụ 10b: WF-GITHUB-RESEARCH (Mode B) — Nghiên cứu repo GitHub để học tập/tham khảo

Tình huống: Người dùng gửi "Nguồn: <https://github.com/some-org/some-lib> — mình muốn tìm hiểu, học tập cách họ làm, chưa cần áp dụng gì vào KZTEK cả".

```
sequenceDiagram
    participant U as User
    participant Disp as Dispatcher
    participant GRR as GitHub Repo Researcher

    U->>Disp: Gửi link GitHub + yêu cầu "học tập/tìm hiểu"
    Disp->>Disp: Có link GitHub, mục đích học tập → kích hoạt WF-GITHUB-RESEARCH (Mode B)
    Disp->>GRR: Giao task (Sonnet)
```



```

GRR->>GRR: Bước 0 – Phase 0 Audit
GRR->>GRR: Bước 1 – Tạo nhánh research/<repo-slug>-<date>, xác định Mode B (user đã
nói rõ mục đích học tập)
GRR->>GRR: Bước 2 – Clone repo vào scratchpad, đọc & phân tích
GRR->>U: Bước 3 – Viết phân tích repo (mục đích, cấu trúc, điểm nổi bật)
GRR->>U: Bước 3c – Hỏi user muốn tìm hiểu tiếp: nguyên lý hoạt động / hướng dẫn áp
dụng-sử dụng / cả hai
U->>GRR: Chọn phần muốn tìm hiểu
loop Đến khi user xác nhận đã nắm rõ
GRR->>U: Bước 3d – Giải thích nguyên lý/cách áp dụng, có ví dụ cụ thể từ repo
nguồn
U->>GRR: Hỏi thêm hoặc xác nhận "đã hiểu rõ"
end
GRR->>GRR: Bước 3e – Viết tài liệu tổng hợp (phân tích + nguyên lý + hướng dẫn áp
dụng), xuất DOCX/PDF
GRR->>U: Xin xác nhận merge nhánh nghiên cứu về main
U->>GRR: Xác nhận merge
GRR->>GRR: Merge về main, báo cáo kết quả

```

Bài học từ ví dụ này:

- Mode B phục vụ đúng nhu cầu học tập/tham khảo — không ép user phải nhận một bảng đề xuất áp dụng KZTEK mà họ không cần.
- Mục tiêu Bước 3d là để **user tự tin nắm rõ nguyên lý, cách vận hành, cách áp dụng** — agent không tự ý chốt tài liệu tổng hợp khi user chưa xác nhận đã hiểu.
- Tài liệu tổng hợp (Bước 3e) vẫn tuân thủ đầy đủ artifact bắt buộc (RESEARCH-*.md + .docx + .pdf, §19 CLAUDE.md) và Git Safety Protocol khi merge — chỉ khác nội dung (nguyên lý/hướng dẫn áp dụng thay vì bảng đề xuất).
- Nếu giữa chừng user đổi ý muốn áp dụng vào KZTEK → agent chuyển sang Mode A (Bước 3b trở đi), không cần bắt đầu lại từ đầu.

Tóm tắt nguyên tắc xuyên suốt

#	Nguyên tắc	Áp dụng khi
---	-----	-----
1	Không nhảy cấp	Mọi lúc, trừ incident SEV1/SEV2
2	2-eyes principle	Mọi review (code, design, deploy)
3	QA có quyền VETO release	Khi còn P0/P1 bug
4	Mitigate trước, fix sau	Production incident
5	Escalate đúng cách	Khi vượt quyền hoặc conflict
6	Junior được phép sai, không được lặp	Mentoring
7	Tài liệu hóa quyết định	Mọi quyết định lớn
8	Khách hàng là trung tâm	Khi có tranh cãi nội bộ
9	UX/UI Reviewer bắt buộc khi đổi giao diện	Trước QA sign-off, nếu code sửa/thêm UI (feature, bugfix, hotfix, fast-track, refactor)
10	Code Migrator chỉ dùng khi được yêu cầu	Không tự động chạy trong bất kỳ workflow nào khác; Opus chỉ dùng ở giai đoạn lập plan/review
11	Song song hoá khi 2 bước độc lập, không quan hệ review	Tăng tốc workflow, không giảm chất lượng kiểm tra (xem

		RULES .md §3.4, ký hiệu // trong CLAUDE .md §4)
12	GitHub Repo Researcher chỉ dùng khi user gửi link GitHub	Không tự động chạy trong workflow khác; hỗ trợ cả 2 mục đích (Mode A cải tiến KZTEK, Mode B học tập/tham khảo); không tự áp dụng đề xuất/chốt tài liệu hay tự merge khi chưa có xác nhận rõ ràng của user